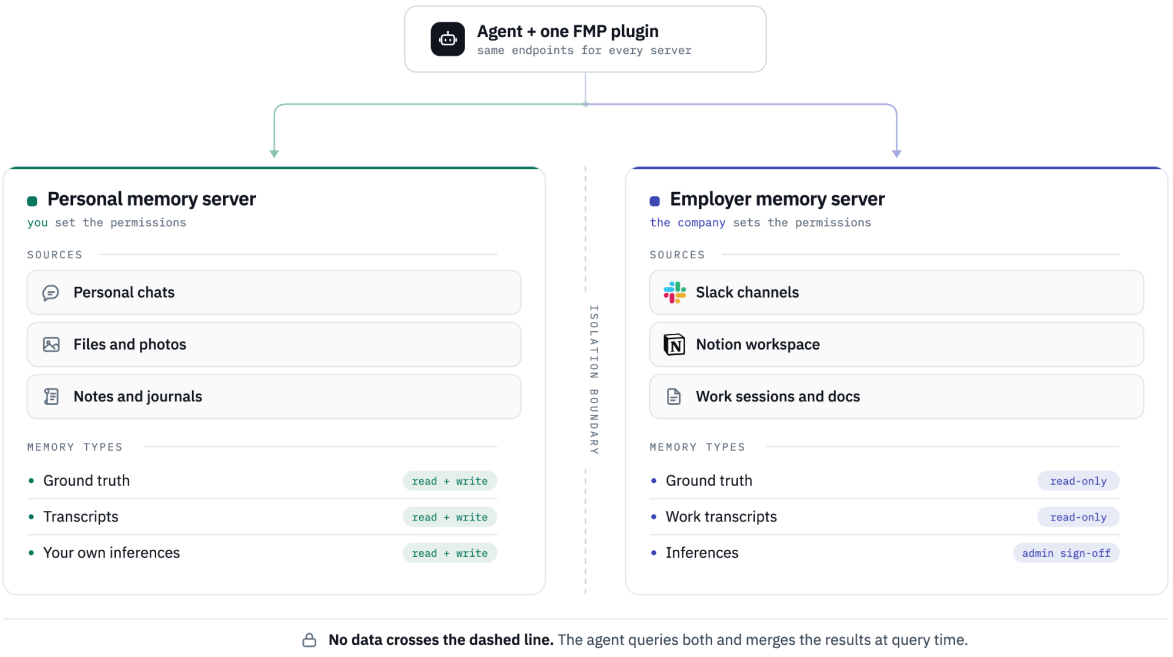
Federated Memory Protocol Draft v1

Sruly Rosenblat, AI Disclosures Project
September 4th

Note: partially inspired by IBMs proposal

Overview

The proposed Federated Memory Protocol (FMP) is a standard interface between agents and memory servers. It is designed so that even very different memory systems can take advantage of the same primitives and harnesses without added work. Instead of looking at a memory server as a full product that supports all the users needs we can instead look at memory as a collection of resources where the user has information stored and allow all the different sources to be accessed in the same way utilising the same tools. The user can still choose to rely on one server for all their data but this is not enforced by the protocol itself.



One FMP plugin can connect to multiple memory servers at once, allowing users to keep their own data private while utilizing both personal and work data to accomplish their tasks.

Implementation details

The protocol is agnostic to how it's delivered. A *memory server* is any backend that exposes the FMP endpoints; it is not the same thing as an MCP server, which is just one way an agent can reach it.

There are three tiers of support, in order of preference:

1. **Native integration.** The harness speaks FMP directly. No plugin or MCP server needed.
2. **Plugin + bundled MCP server.** Where the harness supports plugins, the plugin gains access to hooks (e.g., to capture full transcripts automatically), and the bundled MCP server lets the model search and upload memories on demand.
3. **Plain MCP server.** Where neither of the above is available. The model can still search all memory servers and upload explicit memories, but won't have hooks, so full transcripts likely won't be captured.

In every tier, all connected memory servers expose the same endpoints, so the model can combine them into a single view of the user's history. The protocol says nothing about how a server captures, stores, or searches memory; it only standardizes the interface.

This means that different agents will have different ways of interacting with memory servers:

On supported platforms:

Adding a memory server will be as easy as adding a URL in the settings. The harness itself will deal with the complexity of MCP and Hook setup.

On local coding agents:

The agent will have full access to hooks, ability to upload full transcripts, and gather whatever context you can through a plugin. Plugins only need to be made once for agents by the community but a plugin importantly does not lock you down to one memory backend and one plugin can be used for different memory backends. A bundled MCP server will allow the agent to manually search or manually upload memories.

On more locked down platformers:

Can still search memories from all platforms, can still upload memories to all servers. But will likely not send full transcripts to the servers. MCP is sufficient to gain access to all memories and upload some.

The protocol does not assume a file system. It works on any agent to at least some extent.

Benefits of federation:

A user does not need to give up on their personal accumulated memory to sign up to an employer's server. The user also does not need to upload their private data to a work server. Both memory servers can live side by side connected to one plugin. Users can also easily drop or add a memory server by updating one setting (either a config file or some user interface if better supported).

This setup also allows servers to broaden the idea of memory without competing for another slot as an MCP server in a models context window. All sources of information about a user can be a memory server and the model will be exposed to all of it through one endpoint. Slack can be a memory server without having to support an agent uploading memories to it.

Permissions

Each memory server can handle permissions in their own way, they can provide a URL for users to manually approve or reject memories. They can also allow memories to be written arbitrarily as long as the user is signed in (via OAuth or some other mechanism). Memories can be tagged to different users or different contexts and those contexts can require different permissions within that memory server. Eg, context that will be shared across a company may require sign off from a company administrator. Agent editing can be disabled entirely and only be changed on a users dashboard.

So memory granularity has two levels 1) different servers can refuse different memories and 2) one server can discriminate what memories the agent can upload.

The memory servers will support just three types of memories but those types will be very extensible and hopefully capture everything that can be needed.

Ground truth: Files, discord logs or anything else that is not touched by an AI model prior to being saved. It is not inferences made by an AI about what happened, it is the ground truth the model is using. Not every memory server will expose every type of file. That is fine.

Transcripts: Standardised transcripts or partial transcripts (in case cant access to full one) similar to or using trajectory. Even though this should technically be classified as ground truth I think that it is worth separating because of how important it is in AI, and the ability to potentially link to individual messages in inferences.

Inferences: easily manageable standardized memory inferences with a content field, a date field, a reference to how it was created and exactly what it's based on. TBD exact fields but there should be a nested json to support arbitrary metadata. Inferences can be created by the user's agent through MCP or by a model hosted by the memory provider.

The goal is not to impose how memory is handled, just standardize the interface.

Possible Endpoints

<memory url>/fmp/upload/files
Feed files to the memory system.

<memory url>/fmp/upload/transcript
Feed transcript or partial transcript to memory system

<memory url>/fmp/upload/inferences
Feed inferences to memory system

<memory url>/fmp/search
Search over the entire memory server, optional params to limit to certain types

<memory url>/fmp/read_transcripts
Paginated transcripts for importing chat history

<memory url>/fmp/read_inferences
Paginated transcripts for importing memory inferences

<memory url>/fmp/delete
Give an ID of a memory to delete it

<memory url>/fmp/info
Receive a list of the supported capabilities for the memory server and version info (must always be supported)

Any of these can be disabled by the memory server while still being compliant with the proposed protocol.

Remaining big questions:

- How are memories combined and fed to the model when multiple servers are added? (Should we leave this to the MCP server, should we specify everything should be returned, should we offer the model the chance to filter what is returned)
- How does the model route where a memory is stored, is it an option on a tool call? (Should it be everywhere by default or nowhere by default)
- Are transcripts sent to every memory server on every message? (I imagine this is something that can be easily managed in settings)

Benefits of this approach

- Assumes nothing of either the producer or consumer. As long as MCP is supported, both consumption and explicit memories are supported. If hooks are also supported by a harness then every system connected gets a richer view of memory.
- If there is adoption the same protocol can be integrated directly into the harness so a third party plugin or MCP server is not needed. If it is not integrated any harness can still be supported anyway via MCP and or hooks.
- Is not tied to an operating system. No need for a LLM to be able to read files.

Inspiration:
ARDS, HCP, Cognee, Letta trajectory, IBM memory protocol draft.